Anomaly Detection in Surveillance Videos

Real-Time Computer Vision Pipeline

Mid-Term Progress Report

Seasons of Code — Web and Coding Club, IIT Bombay

RGV Tejesh Yadav

B.Tech. Computer Science and Engineering (First Year)

Indian Institute of Technology Bombay

Mentors: Yogesh Sangwan, James Antil

23rd June 2026

All code, trained weights, datasets, and output videos are maintained in the project repository.

Abstract

Surveillance cameras are everywhere, but nobody actually watches most of the footage. There's simply too much of it. This project is about building a system that does the watching automatically — detecting people in video, tracking them across frames, and flagging behaviour that doesn't fit.

The pipeline has three stages: first, teach a neural network to find pedestrians in every frame using YOLOv5 trained on the MOT17 dataset; second, give each detected person a persistent identity across frames using DeepSORT-based tracking; and third, use those movement trajectories to spot anomalies — someone running when everyone else is walking, moving the wrong way, or lingering somewhere they shouldn't be.

This mid-term report covers the first two stages in full and introduces the third. The theory behind each component is explained from scratch — neural networks, convolutional filters, YOLO's detection strategy, Kalman filtering, and the Hungarian algorithm. Results from training three YOLOv5 variants are compared, and the tracking implementation is shown working on real video.

Contents

1. Introduction — Why Surveillance Needs to Think

Picture a busy railway station. Dozens of cameras. Thousands of people. Somewhere in that crowd, something goes wrong — a bag left near the gate, a person moving against the flow, a scuffle breaking out near an exit. A security team watching sixteen screens at once will almost certainly miss it. The problem isn't the cameras. It's that human attention doesn't scale.

Automated anomaly detection tries to fix this. The idea is to have a computer watch the footage and only surface the moments actually worth a human's attention. It doesn't replace judgment — it acts as a filter. Instead of a person staring at screens for eight hours and catching maybe 30% of incidents, they respond to timestamped alerts and review specific clips.

What makes this genuinely hard is that "normal" looks different in every scene. Running is an anomaly in a hospital corridor but completely expected outside a gym. The system can't just memorise fixed rules — it needs to learn what normal looks like in a given scene and flag whatever doesn't fit. That learning process, from raw pixels to a working decision, is what this project builds.

The pipeline flows through three stages:

- **Detect** — Find every person in every frame.
- **Track** — Follow each person across time, keeping their identity consistent.
- **Reason** — Analyse movement patterns and raise alerts when something's off.

Each stage depends directly on the one before it, which is why understanding the theory matters. If you know how the detector works, you know why the tracker behaves the way it does. And if you understand tracking, the anomaly logic becomes almost obvious.

2. The Brain Behind It — Neural Networks

2.1 What a Neural Network Actually Is

Strip everything down and a neural network is a function. It takes some numbers in, does math to them, and spits other numbers out. That's genuinely all it is. The complexity is in how it learns to do that math well.

The basic unit is a neuron. It takes several inputs, multiplies each one by a weight, adds them up, throws in a bias term to shift things around, and passes the result through a simple non-linear function called an activation function. The activation is usually something like ReLU (Rectified Linear Unit), which just clips negative values to zero. Stack a bunch of these neurons in layers, connect the layers, and you have a neural network.

The input to this network could be anything — the pixel values of an image, a row from a spreadsheet, audio samples. The output could be a class label, a bounding box,

a probability score. What the network learns is the right set of weights to turn one into the other.

Before training, those weights are random. The network's first predictions are garbage. The whole point of training is to fix that.

2.2 How It Learns — Cost, Gradients, and Descent

To know how wrong the network is at any point, we define a **cost function** (also called a loss function). It takes the network's output and the correct answer and produces a single number that measures the gap. High cost = network is wrong. Low cost = network is doing well.

Training is the process of minimising that number. The tool for doing it comes from calculus. For each weight in the network, we compute how much a small change in that weight would change the cost — that's just a derivative. If the derivative is positive, the weight is pulling the cost up, so we decrease it. If it's negative, we increase it.

$$w \leftarrow w - \eta \cdot \frac{\partial \mathcal{L}}{\partial w}$$

Here η is the learning rate — how big a step to take. The process of repeatedly stepping in the downhill direction is **gradient descent**. To compute derivatives for every single weight efficiently, we use **backpropagation**, which applies the chain rule backwards through the network.

The key intuition is this: the error signal flows backward through the layers, telling each weight how much it was responsible for the mistake. Weights that contributed a lot get updated more. Weights that barely mattered get a smaller nudge. Do this for thousands of training examples and the weights slowly converge to something that actually works.

One thing worth saying clearly: the entire trained model is just a file of numbers. After all the training runs, all the compute time, what you actually have is a `.pt` file containing a few million floating-point weights. That file is the model. Load it, feed it an image, get a prediction in milliseconds. Everything else was in service of producing that file.

3. Seeing in Layers — Convolutional Neural Networks

3.1 Why You Can't Just Use a Standard Network on Images

A 640×640 colour image has over 1.2 million individual pixel values. If the first layer of a standard network has 1000 neurons, each connected to every input, you're already looking at over a billion parameters just in that one layer. That's both computationally insane and wasteful — the network would have to independently learn that a vertical edge in the top-left corner looks the same as one in the bottom-right, even though it's obviously the same thing.

Convolutional Neural Networks (CNNs) solve this with two ideas: local connectivity and parameter sharing.

Instead of connecting to every input, each neuron only looks at a small patch of the image (say, a 3×3 region). And instead of each neuron having its own weights, every neuron in a layer shares the same set of weights — called a **filter** or **kernel**. That filter slides across the entire image, computing a dot product at each position. The result is a new grid called a **feature map**.

3.2 What Filters Learn

A filter is just a small grid of learnable numbers. When you slide it across an image and compute dot products, you get a high response wherever the image matches the filter's pattern and a low response elsewhere.

In practice, filters in early CNN layers tend to learn simple detectors — one picks up horizontal edges, another vertical edges, another responds to colour transitions. Go deeper and the filters start combining these into more complex patterns: corners, textures, curves, and eventually parts of objects. The network learns all of this automatically from data. You don't tell it to detect edges. It just finds that this is useful.

After convolution, a **pooling layer** shrinks the feature map by keeping only the strongest activation in each local region (max-pooling). This compresses spatial information while keeping the most important features, and it makes the network slightly robust to small shifts in the image.

[IMAGE: CNN Architecture Diagram]

Insert diagram showing conv layers, pooling, and feature map visualisations

Figure 1: *Progressive feature extraction in a CNN. Early layers detect edges; deeper layers detect object parts.*

3.3 From Classification to Localisation

A plain CNN trained for classification ends with a layer that outputs probabilities: "this image is 92% likely to contain a pedestrian." That's useful, but not enough for surveillance. We need to know *where* in the frame the person is, not just that they're somewhere in it.

Localisation means predicting a bounding box — four numbers that define a rectangle around the detected object: centre-x, centre-y, width, height. These are added as extra outputs alongside the class probabilities, and the network is trained to predict both simultaneously. The same convolutional feature extractor handles both tasks, which is why end-to-end detectors are efficient.

This is also why localisation matters so much here. Tracking requires precise coordinates in every frame. If we only knew "there's a person somewhere in this frame", tracking would be impossible. The bounding box is what gives tracking its anchor.

4. You Only Look Once — The YOLO Architecture

4.1 The Problem with Classic Detectors

Older object detectors worked in two stages: first, generate thousands of candidate regions that might contain an object, then classify each one. Faster R-CNN is the classic example. It's accurate, but it runs at under 10 frames per second. For real-time surveillance, that's useless.

YOLO (You Only Look Once) takes a completely different approach. The whole image is passed through the network in a single forward pass, and in that one pass, the network simultaneously predicts all bounding boxes and class probabilities. No separate proposal stage. One look, full answer. This is why it runs at 45+ frames per second.

4.2 How YOLO Thinks About the Image

Conceptually, YOLO divides the image into an $S \times S$ grid. Each grid cell is responsible for detecting objects whose centre falls within that cell. For each cell, the network predicts several candidate bounding boxes, each with a confidence score and class probabilities.

The confidence score captures two things at once: how certain the network is that an object exists there, and how well-placed the predicted box is around it. A score near 1.0 means confident and tight. A score near 0.1 means the network is basically guessing.

4.3 Anchor Boxes — A Smarter Starting Point

People come in different shapes depending on viewpoint. A standing person seen from the side is tall and narrow. A crowd viewed from above produces compact, roughly square boxes. Getting these proportions right from scratch is hard.

Anchor boxes give the network a head start. Before training, you cluster the ground-truth bounding box shapes in the dataset using k-means. The resulting cluster centres are the anchor shapes. During training, the network doesn't predict boxes from scratch — it predicts how much to adjust each anchor. This is much easier to learn and consistently improves detection of objects with extreme aspect ratios.

4.4 Dealing with Duplicate Detections

YOLO produces thousands of candidate boxes per image. For a single person, multiple nearby grid cells will often each propose a box, all slightly different but all pointing at the same person. This needs to be cleaned up.

The solution uses **Intersection over Union (IoU)**: the ratio of the area where two boxes overlap to the total area they cover together. Two boxes pointing at the same person will have high IoU. Two boxes pointing at different people will have low IoU.

Non-Maximum Suppression (NMS) then works like this: pick the highest-confidence box, discard any other box that overlaps it with IoU above a threshold (we used 0.45), repeat. What comes out is one clean box per detected object, no duplicates.

[IMAGE: IoU and NMS Diagram]

Multiple overlapping boxes before NMS, single box after

Figure 2: *Non-Maximum Suppression. Multiple proposals for the same pedestrian collapse into one high-confidence detection.*

4.5 Model Variants — Picking the Right Size

YOLOv5 comes in several sizes. The trade-off is exactly what you'd expect: bigger model, more accuracy, slower inference.

Model	Params	Speed	Accuracy	Best for
YOLOv5n (Nano)	∼1.9M	Very fast	Low-Medium	IoT, edge devices
YOLOv5s (Small)	∼7.2M	Fast	Medium	Baseline / mobile
YOLOv5m (Medium)	∼21M	Moderate	High	Balanced deployment
YOLOv5l (Large)	∼47M	Slower	Very high	Server-side inference

Table 1: *YOLOv5 model family. This project trained and compared the Small and Medium variants.*

4.6 OpenCV — Reading Video into the Pipeline

A video file is just a sequence of still images played quickly enough to look like motion. **OpenCV** is the library that handles everything around the neural network: reading video files, decoding individual frames, resizing and normalising images to the 640×640 format YOLO expects, and drawing the predicted bounding boxes back onto the frame.

The processing loop is simple: read frame → resize and normalise → run YOLO → apply NMS → draw boxes → write frame. This loop is how you turn a trained neural network into something that works on actual video.

5. Teaching by Example — Dataset and Training

5.1 The MOT17 Dataset

The Multiple Object Tracking 2017 (MOT17) dataset is the standard benchmark for pedestrian detection and tracking. It was specifically designed to be difficult: crowded scenes, moving cameras, night sequences, partial occlusions, pedestrians at various distances. Fourteen video sequences cover environments from shopping malls to night-time town squares.

Every frame comes with ground-truth bounding boxes for every pedestrian, which makes it directly usable as training data for YOLO after some preprocessing.

Sequence	Environment	Main Challenge
MOT17-02	Shopping Mall	Dense crowds, static camera
MOT17-04	Town Square	Night-time, low light
MOT17-09	Pedestrian Street	Low angle, heavy occlusion
MOT17-10	Public Square	Distant pedestrians, small objects

Table 2: *Select MOT17 training sequences used in this project.*

[IMAGE: MOT17 Sample Frames with Ground Truth Annotations]

Frames from the dataset showing annotated pedestrian bounding boxes

Figure 3: *Sample frames from the MOT17 training set. Ground-truth bounding boxes are provided for every pedestrian in every frame.*

5.2 Preprocessing — Getting Annotations into YOLO Format

MOT17 stores bounding boxes as (top-left x, top-left y, width, height) in pixel coordinates. YOLO needs them as (centre-x, centre-y, width, height) normalised to [0, 1] relative to image dimensions. A Python script in Google Colab handles this conversion:

$$x_c = \frac{x_{tl} + w/2}{W}, \quad y_c = \frac{y_{tl} + h/2}{H}, \quad w_{norm} = \frac{w}{W}, \quad h_{norm} = \frac{h}{H} \tag{1}$$

where W, H are the image width and height. After conversion, the data is organised into the folder structure YOLOv5 expects: separate `images/` and `labels/` directories for train, val, and test splits.

5.3 Train / Validation / Test Split

Every machine learning project divides its data into three distinct sets, each serving a different purpose:

- **Training set** — What the network actually trains on. Weights are updated based on this data.
- **Validation set** — Held out from training and used to monitor how well the model generalises during the process. If training loss keeps dropping but validation loss starts rising, the model is memorising rather than learning (overfitting), and training should stop.
- **Test set** — Completely unseen data used for the final evaluation. Think of it as the exam.

Watching the gap between training and validation loss is one of the most useful habits in any ML project. A widening gap almost always signals something going wrong.

5.4 Data Augmentation

More data almost always means better generalisation. But collecting more real surveillance footage is slow and expensive. Augmentation is the alternative: apply random transformations to existing training images to create effective variations.

Augmentations applied during this project include horizontal flips, slight rotations, brightness and contrast jitter, random crops, and colour channel shifts. These transformations are applied on-the-fly during training, so the model rarely sees the exact same image twice. It's one of the cheapest and most effective ways to prevent overfitting.

[IMAGE: Augmented Training Images]

Examples of flipped, rotated, brightness-adjusted training samples

Figure 4: *Data augmentation examples. The same base image appears with different transformations, forcing the model to learn features that hold across variations.*

5.5 Training Parameters

Parameter	Value	What It Controls
Epochs	5	Full passes through the training set
Batch size	16	Images processed per weight update
Learning rate	0.01 (init)	Step size for gradient descent
Image size	640×640	Input resolution fed to the network
Conf. threshold	0.5	Minimum score to keep a detection
IoU thresh (NMS)	0.45	Max overlap before a box is suppressed

Table 3: *Training configuration used across all model variants.*

5.6 The Key Metric — mAP@0.5

Mean Average Precision at IoU threshold 0.5 (mAP@0.5) is the standard metric for object detectors. It measures both how well the detector finds real objects (recall) and how often its detections are actually correct (precision), combined across different confidence thresholds.

A score of 1.0 would mean perfect detection — every pedestrian found, zero false alarms. In practice, 0.70–0.80 on a difficult dataset like MOT17 represents solid performance.

[IMAGE: Training Curves — Loss and mAP vs Epoch]

Paste your results.png from the YOLOv5 training output here

Figure 5: *Training and validation loss along with mAP@0.5 plotted over training epochs.*

[IMAGE: Precision-Recall Curve]

PR curve for YOLOv5m-Frozen on MOT17 validation set

Figure 6: *Precision-Recall curve. The area under this curve (averaged across confidence thresholds) gives the Average Precision.*

5.7 Mask Detection Homework

As a parallel exercise, the same YOLOv5 pipeline was applied to a mask-detection dataset — training the model to localise and classify faces as masked or unmasked. The output bounding boxes from this experiment are shown below.

[IMAGE: Mask Detection Inference Output]

Add bounding box detection outputs from mask dataset here

Figure 7: *Inference results from the mask detection experiment. Each face is localised and classified without any changes to the pipeline — just a different dataset.*

This exercise makes something important clear: the pipeline is largely domain-agnostic. Same architecture, same training loop, same evaluation code — swap the dataset and labels and you get a completely different detector.

5.8 The Weights File

After all the training, what does the model actually look like on disk? A single binary file, usually around 40–170 MB, containing millions of floating-point numbers. That's the weights.

Everything accumulated during training — all of the model's *understanding* of what a pedestrian looks like, across lighting conditions, angles, and distances — is encoded in that one file. Load `best.pt`, feed it an image, get back detections in milliseconds. The training infrastructure, the dataset, the compute time — all of that exists to produce this one file.

6. Detection Results — Comparing Three Models

Three YOLOv5 configurations were trained on the preprocessed MOT17 dataset for 5 epochs each:

Model	Params	Precision	Recall	mAP@0.5	Train time
YOLOv5s (Baseline)	7.2M	0.821	0.610	0.725	∼0.10 hrs
YOLOv5m (Full)	20.85M	0.874	0.691	0.785	∼0.28 hrs
YOLOv5m (Frozen)	20.85M	0.859	0.662	0.773	∼0.17 hrs

Table 4: *Performance comparison of the three YOLOv5 variants trained on MOT17. The Frozen variant freezes the first 10 backbone layers during training.*

A few things worth noting from these results:

The full YOLOv5m got the highest mAP (0.785), as expected — more parameters, more expressive power. But the frozen YOLOv5m is actually the better practical choice. It gets 0.773 mAP with training time nearly halved. The reason is transfer learning.

The backbone of YOLOv5m was pre-trained on the COCO dataset, which has over 120,000 images and 80 different object categories. That backbone already knows how to extract useful visual features. Freezing it during fine-tuning on MOT17 prevents the network from overwriting those learned features with noise — a phenomenon called catastrophic forgetting. The head (which interprets features into detections) adapts to pedestrians while the backbone stays general and useful.

[IMAGE: YOLOv5m (Frozen) Qualitative Detections]

High-confidence bounding boxes on crowded MOT17 validation frames

Figure 8: *Detection output from the frozen YOLOv5m model on the MOT17 validation set.*

[IMAGE: Pedestrian Dataset Homework Output]

Output images from the MOT17 pedestrian homework notebook

Figure 9: *Inference outputs from the MOT17 homework exercise, submitted separately in the project repository.*

7. Following the Story — Multi-Object Tracking

7.1 Why Detection Isn't Enough

The detector is now reliably finding people in individual frames. But anomaly detection needs more than a snapshot — it needs a narrative. Is that person walking toward the exit or away from it? Have they been standing in the same spot for ten minutes? Did they just break into a run?

None of these questions can be answered by looking at one frame. You need the whole sequence, and for that, each person needs a persistent identity across time. That's what multi-object tracking (MOT) does.

7.2 The Tracking-by-Detection Approach

This project uses tracking-by-detection: run the detector independently on every frame, then solve the problem of matching each new detection to the existing tracks from the previous frame. This decouples the detector from the tracker cleanly — you can swap either one without breaking the other.

The core challenge is that the matching isn't trivial. People move, overlap, disappear behind objects, and reappear. Simple nearest-neighbour matching breaks immediately in any crowded scene.

7.3 The Kalman Filter — Predicting Motion

The **Kalman Filter** is a classical algorithm that tracks the state of a moving object under uncertainty. It models each tracked person's state as position and velocity, and maintains a statistical estimate of where they are and how confident it is.

It works in two alternating steps:

1. **Predict** — Using the current velocity estimate, project where the person will be in the next frame. This gives an expected position even before the detector runs.
2. **Update** — When YOLO produces a detection, blend the prediction with the measurement. The filter automatically weights each based on how trustworthy it is.

The practical consequence is important: if a pedestrian walks behind a pillar for several frames and the detector sees nothing, the Kalman Filter keeps their track alive based on predicted motion. When they reappear, the same ID is maintained. Without this, every occlusion permanently kills the track.

7.4 The Hungarian Algorithm — Assigning Detections to Tracks

After the Kalman Filter predicts where each existing track expects to be, and YOLO has produced new detections, there's a combinatorial problem: if there are 20 existing tracks and 20 new detections, which detection belongs to which track?

The **Hungarian Algorithm** finds the optimal one-to-one assignment. It builds a cost matrix where entry (i, j) is the IoU distance between predicted track i and detection j, then finds the assignment that minimises total cost. The solution is exact and runs in polynomial time.

Unmatched detections become new tracks. Tracks with no matching detection get aged; if they stay unmatched long enough, they're deleted.

7.5 DeepSORT — Adding Visual Memory

Simple SORT (Kalman + Hungarian) works well for short occlusions. But when someone disappears for a long time, the position prediction drifts and IoU matching fails.

DeepSORT adds a visual appearance model. A small neural network extracts a compact feature vector (128 dimensions) from each detected bounding box crop. These embeddings are stored for each track as an *appearance gallery*. When matching, the cost function now blends two signals:

- **Motion cost**: Mahalanobis distance between the Kalman-predicted position and the detection, accounting for prediction uncertainty.
- **Appearance cost**: Cosine distance between stored appearance embeddings and the new detection's features.

The combination makes re-identification after long occlusions significantly more reliable.

7.6 BoT-SORT — What Was Used in This Project

This project used **BoT-SORT** (Box Tracking with SORT), which extends DeepSORT with camera motion compensation. When the camera is moving, a pedestrian's displacement in the frame is a mix of their actual movement and the camera's. BoT-SORT estimates and removes the camera component, making tracking noticeably more stable in non-static camera scenarios.

Implementation was straightforward using the Ultralytics library:

```
model = YOLO('best.pt')
results = model.track(
    source="mot_sample.mp4",
    conf=0.5,
    persist=True,
    classes=[0]   # Person class only
)
```

The `classes=[0]` filter came from a practical problem: the COCO-pretrained backbone occasionally produced false positives on background objects like monitors and furniture. Restricting output to class 0 (Person) eliminated this completely without any retraining.

[IMAGE: Tracking Output — Before Filtering]

Initial tracking showing false positives on background objects

Figure 10: *Initial tracking output before class filtering. Pedestrian IDs are maintained, but the pre-trained backbone misidentifies some background elements.*

[IMAGE: Tracking Output — After Filtering (classes=[0])]

Clean tracking with only person class detected

Figure 11: *After applying* `classes=[0]`*. Environmental noise eliminated; pedestrian identity persists through occlusions and path crossings.*

[IMAGE: Road Video Tracking — Sample Frame]

Frame from road tracking video showing multi-object tracking on real footage

Figure 12: *Still frame from the road scene tracking video. Multiple objects tracked simultaneously with persistent IDs across the full sequence.*

8. When Normal Breaks Down — Anomaly Detection

8.1 What Counts as an Anomaly

An anomaly is anything that deviates from what the system has learned to expect. In a surveillance context, the system builds a model of what normal behaviour looks like in that specific scene — the typical speed of movement, dominant directions, crowd density, which zones people use. Anything that doesn't fit gets flagged.

Some concrete examples of detectable anomalies:

- A person moving significantly faster than everyone around them
- Someone moving against the dominant crowd flow
- A stationary figure in an area where people normally keep moving
- Sudden dispersal of a previously calm crowd
- Two tracks merging and one disappearing (potential altercation)
- Entry into a defined restricted zone

8.2 Three Approaches to Learning Normality

Approach	Training Data	How Anomalies Are Found
Supervised	Labelled normal + anomaly examples	Directly classifies each event
Unsupervised	Normal examples only	Flags deviations from learned distribution
Semi-supervised	Mostly normal, few anomalies	Hybrid of both signals

Table 5: *Anomaly detection approaches. This project uses an unsupervised pipeline.*

This project uses unsupervised detection: train on normal pedestrian behaviour from MOT17, and flag whatever deviates from that. This is the most practical approach because anomalies are by definition rare and hard to predefine. You can't collect a labelled dataset of every possible unusual event.

8.3 Rule-Based Logic on Tracked Trajectories

Given the trajectory data from the tracker — each person's (x, y) position in every frame — a set of rules can identify anomalies without any additional learning:

- **Speed threshold**: If frame-to-frame displacement exceeds some multiple of the median speed across all tracks, flag as running.
- **Direction check**: Compute the dominant crowd flow direction from average velocity vectors. Flag anyone moving more than 90° against that direction.
- **Dwell time**: Flag tracks that stay stationary in a normally-moving zone for more than a set duration.
- **Zone violations**: Define polygonal restricted areas on the camera image. Any entry triggers an alert.
- **Crowd density spike**: If the number of active tracks in a region increases sharply within a short window, flag as a crowd surge.

8.4 Learning-Based Methods

For more complex cases, purely rule-based logic isn't enough. The research literature has several approaches:

Autoencoder-based detection. Train an autoencoder (a network that compresses then reconstructs its input) only on normal frames. When an anomalous frame arrives, the network struggles to reconstruct it accurately — reconstruction error spikes. The error signal itself becomes the anomaly score.

Trajectory prediction models. Train an LSTM or transformer to predict where each person will be in the next N frames given their history. If the actual future position deviates sharply from the prediction, something unexpected happened.

One-class SVM / Deep SVDD. Learn a hypersphere in feature space that tightly

encloses representations of normal events. Anomalies land outside the sphere; their distance from the boundary is the anomaly score.

GAN-based normalcy modelling. A Generative Adversarial Network learns to generate realistic-looking normal video. Frames the generator cannot reproduce well signal anomalies.

The thread connecting all of these: build an accurate model of what normal looks like, then flag whatever the model can't explain. The better your model of normal, the sharper your definition of abnormal.

8.5 What This Technology Can Actually Do

It's worth being direct about the real-world scope here, because it's genuinely large. The same detect-track-reason pipeline that catches a person running in the wrong direction can, with the right detector with applied logic and some changes with training can do a very vast things which are highly useful some of them are :

- **Criminal identification**: Cross-reference face or gait signatures against watchlists. Flag escaped convicts or persons of interest in real time across a city's camera network.
- **Vehicle tracking**: Detect and track stolen vehicles, monitor parking violations, catch wrong-way drivers on highways.
- **Fire and smoke detection**: Swap the pedestrian detector for a fire/smoke model. Track spread trajectories and trigger evacuation protocols.
- **Robbery and assault detection**: When someone robbed ur home when no one is there it can alert automaatically like modern security systems.Rapid directional movement toward another person followed by their collapse or flight triggers an immediate alert.
- **Crowd management at scale**: Monitor concert or stadium crowds for dangerous density thresholds and stampede precursors before the situation becomes unmanageable.
- **Healthcare monitoring**: Detect patient falls, unusual movement patterns, or unauthorised ward access in hospitals and care homes without continuous manual monitoring.

All of these use the same architecture. Different detector, different reasoning rules, same pipeline. The modular design is what makes the technology generalisable.

9. Conclusion and What Comes Next

Over the past four weeks, this project went from a baseline study of neural networks to a fully functioning, real-time pedestrian detection and tracking system. By combining a fine-tuned YOLOv5m detector with DEEP-SORT, the final pipeline doesn't just spot pedestrians in a video stream; it actually locks onto them and maintains consistent IDs across frames.

One of the biggest takeaways was how effective transfer learning can be. By freezing the backbone network, I managed to get the best of both worlds: training time dropped to around small time compared to other models, but the accuracy held up just as well as if I had fine-tuned the entire model from scratch. I also realized that you don't always need to solve deployment issues by throwing more training data at them; sometimes, smart system design is enough. For instance, the model initially picked up a lot of random objects because of its COCO pretraining, but simply filtering the output to focus exclusively on the pedestrian class (classes=[0]) wiped out those false positives instantly.

In terms of performance, the system hit an mAP@0.5 score of 0.773 on detection and generated clean, reliable tracking data that's ready for next-stage analysis like behavior monitoring or anomaly detection. Ultimately, this project was about a lot more than just writing code—it was a hands-on crash course in prepping datasets, training and evaluating models, tuning hyperparameters, and figuring out how to piece different computer vision components together into a single, cohesive pipeline.

9.1 Limitations and Applications:

Although this can be used in real life for many uses with decent accuracy it has some limitations like it cant identify some anomaly cases as all of them cant be covered possibly and it needs logical thinking for itself to be able to know what is happening people can get their ways to surpass the cameras with non suspicious activities so some new methods should be invented if we want to optimise the performance and safety.Although it can give false negatives which are a kid running just as fun towards another person in mall which was not seen by the model so it might report some false normal scenes too but with small amount of training it is still very useful so a human assist with it would be very progressive than most of the present methods like pure human labour.

What's left:

- Implement speed, direction, and dwell-time anomaly flags on the tracker's trajectory output.
- Train a basic autoencoder on normal MOT17 frames and test reconstruction error on anomalous cases.
- Build an end-to-end demo video: detection → tracking → anomaly flag → visual alert overlay.
- Evaluate with precision/recall on a held-out anomaly test set.

Looking back at what seemed like a complex project from the outside, the individual components are actually intuitive. A network that learns from examples. A detector that looks everywhere at once.It's just a sensible stack of tools, each solving one piece of a well-defined problem.

All code, trained weights, notebooks, and output videos are available in the project repository.